Introduction

When designing and debugging a RISC-V CPU core, I encountered a significant challenge: with the growth of the design scale, the traditional debugging method of reading the dumped waveform became extremely time-consuming and inefficient. While tracing a simulation with millions of cycles, locating a design flaw from the huge waveform file became an impossible mission. This experience urged me to develop a simulation framework inspired by GEM5. The framework can simulate the design with a core-state monitor, and perform a difference-test compared with industry-level RISC-V emulators (including QEMU and Spike) on the simulation process and result. This framework has significantly boosted the efficiency of debugging the design and made me realize that the importance of improving hardware design methodology and tool-chain far exceeds the scope of any single project.

I started committing myself to **Electronic Design Automation (EDA)** and **Computer Architecture** based on this perception. In the following academic journey, I was fortunate to participate in several original projects: from optimizing and automating RTL model generation to designing next-generation hardware design language. These experiences have not only strengthened my technical abilities but have also reinforced my resolve to advance innovation in hardware design methodologies. My goal is to make hardware design more efficient and reliable by improving EDA workflows and developing open-source hardware communities. The University of Michigan CSE's excellence in computer architecture and EDA, along with the alignment between my research interests in RTL optimization and Prof. Nathaniel Bleier's work, makes me confident that this is the ideal platform to pursue my research goals.

Academic & Research Background

My research journey began with Project "One Core for Life," where I implemented a fully functional and tape-out ready RISC-V 64IM processor core with a 5-stage pipeline and three-level cache architecture. Using Chisel for the design allowed me to master essential processor design concepts while gaining hands-on experience with modern hardware design tool-chains. The simulation framework I developed supported both customized user programs and the official RISC-V Foundation test suite, enabling thorough functionality testing from unit cases to complex system-level verification. Through this project, I mastered the complete workflow from high-level hardware construction language (Chisel) to RTL synthesis.

At Princeton Inspired by this experience, I continued research on optimizing RTL models during my research internship at Princeton University with Dr. Yu Zeng. We identified a significant efficiency bottleneck in traditional RTL generators, where the RTL models generated by compilers usually contain massive redundancies for cycle-accurate simulation, leading to significantly low simulation efficiency. To solve this problem, we completed two key works: first, we extended the code generator of FIRRTL's compiler, especially focus-

ing on the performance of the Dead Code Elimination (DCE) algorithm when processing memory modules; then, we developed a code pass for the Yosys synthesizer, creatively implementing the Constraint Propagation technique into the generation of RTL modules. I set up experimental test benches on various Chisel-based hardware designs, including CPU cores, FPUs, and domain-specific accelerators, to test the RTL modules generated by our approach and compared the results with the baselines. The results showed a 2x simulation time acceleration. This work led to a publication in ICCAD'24. More importantly, it proved a key idea that it is feasible to apply mature software engineering optimization approaches to hardware design workflows. This discovery greatly impacted my thinking about innovation in hardware design methodology.

At Georgia Institute of Technology During my internship at EIC Lab,Georgia Tech, I participated in a project focusing on optimizing LLM fine-tuning on edge devices. I built an analytical model based on FlexGen to optimize the computation scheduling diagram and data distribution inside the edge GPU's memory hierarchy. During this practical experience, I discovered that the internal architecture of GPUs still contains many unknown details compared to CPUs, which led to only superficial optimization, despite GPUs have being well-studied and widely utilized. This work was published in DAC'24, yet it made me certain that rather than making incremental improvements to existing architecture, rethinking and rebuilding hardware design methodologies fundamentally is the key. This belief pushed me to move on to a more challenging topic during my experience at KAUST: reconsidering the nature of hardware design language.

At KAUST In the project Assassyn, we were focusing on resolving a basic challenge in hardware design: traditional hardware design and languages enforce developers to handle the details of complex timing and parallelism at the RTL level, which certainly adds burden during development and can lead to mistakes. To address this challenge, we proposed an event-driven and unified abstraction, which could significantly simplify the hardware design flow while maintaining performance.

As one of the core developers of project Assassyn, I was in charge of designing and maintaining the Python-based language frontend. During development, we faced a critical challenge: how could we create a language with an engineering-friendly interface while keeping it highly expressive? After multiple iterations and refactoring, I developed a frontend with full IDE support and debugging capabilities. This new frontend not only provides intuitive error messages and code completion but also handles hardware-specific complex issues, including bit-width inference and combinational logic validation, successfully.

This project has led to encouraging results: our approach both reduces the code size by about 64% and achieves significantly improved performance compared to multiple baseline tests. By implementing various complex designs, including a CPU core, using Assassyn, we have proven the generality of this high-level abstraction. This work is currently under

submission to ISCA'25, and we believe it opens up new possibilities for the next generation hardware design toolchain.

Research Interests & Program Fit

Based on my research experience and understanding of the area currently, my future research will focus on two relevant topics: advancing the evolution of hardware design languages and improving existing EDA toolchains.

For hardware design languages, I plan to conduct in-depth research on porting and applying mature software engineering abstractions to hardware design. While the project Assassyn demonstrates the accessibility of the event-driven model, it is only a starting point. I would like to discover more intuitive and efficient hardware design methods, especially concerning state management and parallelism. Meanwhile, I am also interested in formal verification, focusing on maintaining correctness at this new level of abstraction.

For the EDA toolchain, I hope to continue my research topic started at Princeton by applying compiler optimization techniques to the hardware design workflow. Specifically, I plan to conduct research on advanced algorithms that optimize RTL models based on design intent during the logic synthesis stage. This aligns well with Professor Nathaniel Bleier's research in RTL optimization, particularly his work using Don't Care information to optimize RTL model synthesis. Under his mentorship, I believe we can achieve groundbreaking advances in this direction. Even more exciting is his contribution to space computation, which demonstrates how we as hardware architects can push the boundaries of human discovery—this is precisely why I immerse myself in hardware architecture research.

To pursue these research goals, the University of Michigan's Computer Science & Engineering Ph.D. program stands out as the ideal platform. First, it has a world-class research team, particularly in computer architecture and EDA, providing the perfect environment for cross-disciplinary innovation. The program excels in combining EDA with reliability research, developing domain-specific hardware architectures for biomedical applications such as genomics computation, and integrating chip design with machine learning advances. Additionally, the University of Michigan is actively involved in developing the open-source hardware community, which perfectly aligns with my desire to advance open-source EDA development.